

Export Kode Notebook dan Aplikasi Streamlit

Proyek UAS PBA ABSA Multilabel & NER - Kelompok 2

```
=====
FILE: notebook/Kelp2_multilabel_modeling.ipynb
=====
```

```
-----
CELL 001 [MARKDOWN]
```

```
# Kelp2 - UAS PBA: Multilabel Text Classification ABSA
```

Notebook ini mengerjakan **Proyek A - Multilabel Text Classification ABSA** dari review Google Places. Tujuannya adalah memprediksi satu atau lebih label ABSA pada level review, misalnya ``PRODUCT_POSITIVE``, ``PRICE_POSITIVE``, ``PLACE_NEGATIVE``, ``PROMOTION_NEUTRAL``, atau ``OUT_OF_TOPIC``.

Notebook sengaja dibuat bertahap seperti guided notebook:

1. data loading dan validasi dataset,
2. preprocessing dan label engineering,
3. EDA dan visualisasi label,
4. feature engineering / representasi teks,
5. training beberapa model dan tuning sederhana,
6. evaluasi test set,
7. confusion matrix per label,
8. error analysis,
9. penyimpanan model terbaik,
10. contoh inferensi dan hubungan dengan output NER.

```
-----
CELL 002 [MARKDOWN]
```

```
## 0. Pemetaan terhadap ketentuan soal UAS
```

Bagian notebook ini disusun langsung mengikuti rubrik Proyek A:

```
| Kriteria UAS | Implementasi pada notebook |
|---|---|
| Data loading dan validasi | Membaca `Kelp2_multilabel_train.csv`, `val`, `test`, menampilkan contoh data, memvalidasi kolom teks dan label 0/1. |
| Preprocessing dan label engineering | Cleaning teks, normalisasi huruf kecil, encoding multilabel dari kolom label biner, dan menggunakan split train/validation/test. |
| EDA dan visualisasi | Distribusi panjang review, jumlah token, distribusi label ABSA, dan correlation matrix antar label. |
| Representasi teks | Membandingkan TF-IDF word unigram-bigram dan Bag of Words unigram-bigram. |
| Model training | Membandingkan Binary Relevance/OneVsRest dan Classifier Chain dengan Logistic Regression dan Linear SVM. |
| Evaluasi | Classification report per label, micro/macro/weighted-F1, Hamming Loss, subset accuracy, dan confusion matrix per label. |
| Deployment | Model terbaik disimpan dalam `.joblib` dan dipakai oleh aplikasi Streamlit. |
-----
```

```
CELL 003 [MARKDOWN]
```

```
## 1. Import library dan konfigurasi path
```

Cell ini menyiapkan library utama. ``scikit-learn`` digunakan untuk pemodelan multilabel karena ringan, mudah direplikasi, dan cocok untuk baseline yang kuat pada dataset teks berukuran kecil-menengah. Path dibuat fleksibel supaya notebook tetap bisa dijalankan dari folder ``project/`` maupun dari folder ``project/notebook/``.

```
-----
CELL 004 [CODE]
```

```
from pathlib import Path
import re
import json
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier
from sklearn.multioutput import ClassifierChain
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    classification_report,
    f1_score,
    hamming_loss,
    accuracy_score,
    confusion_matrix,
)

# Agar path aman ketika notebook dijalankan dari project/ atau project/notebook/
CWD = Path.cwd()
if (CWD / 'dataset').exists():
    ROOT = CWD
elif (CWD.parent / 'dataset').exists():
    ROOT = CWD.parent
else:
    ROOT = Path('.')

DATA_DIR = ROOT / 'dataset'
RESULTS_DIR = ROOT / 'results'
```

```
MODELS_DIR = ROOT / 'models'
FIGURES_DIR = ROOT / 'figures'
```

```
for p in [RESULTS_DIR, MODELS_DIR, FIGURES_DIR]:
    p.mkdir(parents=True, exist_ok=True)
```

```
print('Project root:', ROOT.resolve())
print('Dataset dir :', DATA_DIR.resolve())
```

CELL 005 [MARKDOWN]

2. Skema label ABSA

Dataset UAS memakai **13 label multilabel**. Empat aspek utama adalah `PRODUCT`, `PRICE`, `PLACE`, dan `PROMOTION`; masing-masing aspek dapat memiliki sentimen `POSITIVE`, `NEGATIVE`, atau `NEUTRAL`. Label `OUT\_OF\_TOPIC` digunakan saat review tidak relevan dengan empat aspek tersebut.

CELL 006 [CODE]

```
LABELS = [
    'PRODUCT_POSITIVE', 'PRODUCT_NEGATIVE', 'PRODUCT_NEUTRAL',
    'PRICE_POSITIVE', 'PRICE_NEGATIVE', 'PRICE_NEUTRAL',
    'PLACE_POSITIVE', 'PLACE_NEGATIVE', 'PLACE_NEUTRAL',
    'PROMOTION_POSITIVE', 'PROMOTION_NEGATIVE', 'PROMOTION_NEUTRAL',
    'OUT_OF_TOPIC'
]

label_info = pd.DataFrame({
    'label': LABELS,
    'aspect': [x.split('_')[0] if x != 'OUT_OF_TOPIC' else 'OUT_OF_TOPIC' for x in LABELS],
    'sentiment': [x.split('_')[1] if x != 'OUT_OF_TOPIC' else '-' for x in LABELS],
})
display(label_info)
```

CELL 007 [MARKDOWN]

3. Data loading train/validation/test

Cell ini membaca dataset split untuk klasifikasi multilabel. Split sudah disediakan agar eksperimen konsisten antara model dan agar evaluasi akhir hanya dilakukan pada test set.

CELL 008 [CODE]

```
train_df = pd.read_csv(DATA_DIR / 'Kelp2_multilabel_train.csv')
val_df = pd.read_csv(DATA_DIR / 'Kelp2_multilabel_val.csv')
test_df = pd.read_csv(DATA_DIR / 'Kelp2_multilabel_test.csv')

split_summary = pd.DataFrame({
    'split': ['train', 'validation', 'test'],
    'jumlah_review': [len(train_df), len(val_df), len(test_df)],
    'jumlah_kolom': [train_df.shape[1], val_df.shape[1], test_df.shape[1]],
})
display(split_summary)

display(train_df.head(5))
```

CELL 009 [MARKDOWN]

4. Validasi struktur dataset

Validasi dilakukan sebelum modeling supaya error data tidak ikut terbawa ke model. Hal yang dicek:

- kolom `text` harus ada,
- semua kolom label harus ada,
- nilai label harus biner 0/1,
- tidak ada teks kosong,
- setiap review minimal memiliki satu label.

CELL 010 [CODE]

```
def validate_multilabel_dataframe(df: pd.DataFrame, split_name: str) -> dict:
    missing_columns = [c for c in ['text'] + LABELS if c not in df.columns]
    if missing_columns:
        raise ValueError(f'{split_name}: kolom hilang: {missing_columns}')

    # Cek nilai label hanya 0/1
    invalid_label_values = {}
    for label in LABELS:
        values = set(df[label].dropna().unique().tolist())
        if not values.issubset({0, 1}):
            invalid_label_values[label] = values
    if invalid_label_values:
        raise ValueError(f'{split_name}: nilai label tidak biner: {invalid_label_values}')

    empty_text = df['text'].isna().sum() + (df['text'].astype(str).str.strip() == '').sum()
    rows_without_label = (df[LABELS].sum(axis=1) == 0).sum()

    return {
        'split': split_name,
        'rows': len(df),
        'empty_text_rows': int(empty_text),
        'rows_without_label': int(rows_without_label),
        'duplicate_text_rows': int(df['text'].duplicated().sum()),
    }

validation_report = pd.DataFrame([
    validate_multilabel_dataframe(train_df, 'train'),
```

```

        validate_multilabel_dataframe(val_df, 'validation'),
        validate_multilabel_dataframe(test_df, 'test'),
    ])
    display(validation_report)

```

CELL 011 [MARKDOWN]

5. Preprocessing teks

Review Google Places biasanya mengandung huruf besar-kecil tidak konsisten, tanda baca berlebih, angka, URL, emoji, dan spasi ganda. Cleaning yang digunakan bersifat konservatif agar makna review tetap terjaga:

- lowercase,
- hapus URL,
- hapus karakter selain huruf/angka/spasi/underscore/hyphen,
- rapikan spasi.

Stopword tidak dihapus karena kata pendek seperti `tidak`, `ga`, `kurang`, dan `murah` penting untuk sentimen.

CELL 012 [CODE]

```

def clean_text(text: str) -> str:
    text = str(text).lower()
    text = re.sub(r'http\S+|www\.\S+', ' ', text)
    text = re.sub(r'^\0-9a-zA-Z_\-\s+', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

for df in [train_df, val_df, test_df]:
    df['text_clean'] = df['text'].apply(clean_text)
    df['char_length'] = df['text_clean'].str.len()
    df['word_count'] = df['text_clean'].str.split().apply(len)

comparison = train_df[['text', 'text_clean']].head(5)
display(comparison)

```

CELL 013 [MARKDOWN]

6. Label engineering dan penyiapan X-y

Format multilabel diubah menjadi matriks biner `y` berukuran `(jumlah\_review, jumlah\_label)`. Satu review bisa memiliki lebih dari satu nilai 1, misalnya review yang memuji produk sekaligus harga.

CELL 014 [CODE]

```

X_train = train_df['text_clean'].tolist()
X_val = val_df['text_clean'].tolist()
X_test = test_df['text_clean'].tolist()

y_train = train_df[LABELS].astype(int).values
y_val = val_df[LABELS].astype(int).values
y_test = test_df[LABELS].astype(int).values

print('X_train:', len(X_train), '| y_train:', y_train.shape)
print('X_val :', len(X_val), '| y_val :', y_val.shape)
print('X_test :', len(X_test), '| y_test :', y_test.shape)

label_cardinality = y_train.sum(axis=1).mean()
label_density = label_cardinality / len(LABELS)
print(f'Label cardinality train: {label_cardinality:.3f}')
print(f'Label density train : {label_density:.3f}')

```

CELL 015 [MARKDOWN]

7. Strategi penanganan label imbalance

Dataset ABSA biasanya tidak seimbang: label positif produk/tempat jauh lebih banyak daripada label promosi atau netral. Strategi yang digunakan:

1. memakai `class\_weight='balanced'` pada Logistic Regression dan Linear SVM,
2. mengevaluasi dengan macro-F1 selain micro-F1 agar label minoritas ikut terlihat,
3. menampilkan confusion matrix per label untuk melihat false positive dan false negative tiap label.

CELL 016 [CODE]

```

label_distribution = train_df[LABELS].sum().sort_values(ascending=False).reset_index()
label_distribution.columns = ['label', 'positive_count_train']
label_distribution['percentage_train'] = label_distribution['positive_count_train'] / len(train_df) * 100
display(label_distribution)

```

CELL 017 [MARKDOWN]

8. EDA 1 - Distribusi panjang review

Visualisasi panjang review membantu menentukan representasi teks dan batasan fitur. Review pendek biasanya hanya memuat satu aspek, sedangkan review panjang dapat berisi banyak aspek sekaligus.

CELL 018 [CODE]

```

plt.figure(figsize=(8, 4))
plt.hist(train_df['char_length'], bins=30)
plt.title('Distribusi Panjang Review (Karakter) - Train')
plt.xlabel('Jumlah karakter setelah cleaning')
plt.ylabel('Jumlah review')
plt.tight_layout()
plt.savefig(FIGURES_DIR / 'review_length_distribution.png', dpi=150)
plt.show()

```

```
train_df[['char_length', 'word_count']].describe()
```

CELL 019 [MARKDOWN]

9. EDA 2 - Distribusi jumlah kata/token

Jumlah token digunakan untuk melihat variasi panjang teks. Pada pendekatan Bow/TF-IDF, token yang terlalu jarang biasanya tidak terlalu membantu, sehingga `max\_features` dibatasi agar model tidak terlalu sparse.

CELL 020 [CODE]

```
plt.figure(figsize=(8, 4))
plt.hist(train_df['word_count'], bins=30)
plt.title('Distribusi Jumlah Kata/Token - Train')
plt.xlabel('Jumlah token')
plt.ylabel('Jumlah review')
plt.tight_layout()
plt.show()
```

CELL 021 [MARKDOWN]

10. EDA 3 - Distribusi label ABSA

Grafik ini menunjukkan label mana yang dominan dan label mana yang minoritas. Informasi ini dipakai saat membaca hasil macro-F1 dan error analysis.

CELL 022 [CODE]

```
plt.figure(figsize=(10, 5))
train_df[LABELS].sum().sort_values(ascending=False).plot(kind='bar')
plt.title('Distribusi Label ABSA - Train')
plt.xlabel('Label ABSA')
plt.ylabel('Jumlah review positif untuk label')
plt.tight_layout()
plt.savefig(FIGURES_DIR / 'multilabel_label_distribution.png', dpi=150)
plt.show()
```

CELL 023 [MARKDOWN]

11. EDA 4 - Correlation matrix antar label

Correlation matrix membantu membaca hubungan antar label. Misalnya, review yang memuji tempat bisa saja sekaligus memuji produk. Korelasi tidak digunakan langsung sebagai fitur, tetapi berguna untuk menjelaskan mengapa Classifier Chain dapat bermanfaat karena mempertimbangkan dependensi antar label.

CELL 024 [CODE]

```
corr = train_df[LABELS].corr().fillna(0)
plt.figure(figsize=(9, 8))
plt.imshow(corr, aspect='auto')
plt.xticks(range(len(LABELS)), LABELS, rotation=90, fontsize=7)
plt.yticks(range(len(LABELS)), LABELS, fontsize=7)
plt.colorbar()
plt.title('Correlation Matrix Antar Label ABSA')
plt.tight_layout()
plt.savefig(FIGURES_DIR / 'label_correlation_matrix.png', dpi=150)
plt.show()
```

```
display(corr.round(2))
```

CELL 025 [MARKDOWN]

12. Feature engineering / representasi teks

Dua representasi teks dibandingkan:

1. ****TF-IDF word unigram-bigram****
Memberi bobot lebih tinggi pada kata/frasa yang khas dan lebih rendah pada kata yang terlalu umum.
2. ****Bag of Words unigram-bigram****
Menggunakan frekuensi kemunculan token/frasa. Representasi ini sederhana tetapi sering kuat pada dataset review.

Keduanya memakai n-gram `(1,2)` agar frasa seperti `harga murah`, `tempat nyaman`, dan `bahan bagus` dapat tertangkap.

CELL 026 [CODE]

```
def make_tfidf_logreg(C: float = 1.0) -> Pipeline:
    return Pipeline([
        ('vectorizer', TfidfVectorizer(max_features=6000, ngram_range=(1, 2), min_df=1)),
        ('classifier', OneVsRestClassifier(
            LogisticRegression(max_iter=1000, class_weight='balanced', C=C, solver='liblinear')
        )),
    ])

def make_tfidf_svm(C: float = 1.0) -> Pipeline:
    return Pipeline([
        ('vectorizer', TfidfVectorizer(max_features=6000, ngram_range=(1, 2), min_df=1)),
        ('classifier', OneVsRestClassifier(
            LinearSVC(class_weight='balanced', C=C, max_iter=3000)
        )),
    ])

def make_bow_chain_logreg(C: float = 1.0) -> Pipeline:
```

```

return Pipeline([
    ('vectorizer', CountVectorizer(max_features=6000, ngram_range=(1, 2), min_df=1)),
    ('classifier', ClassifierChain(
        LogisticRegression(max_iter=1000, class_weight='balanced', C=C, solver='liblinear'),
        order='random',
        random_state=42,
    )),
])

```

CELL 027 [MARKDOWN]

13. Fungsi evaluasi multilabel

Evaluasi menggunakan beberapa metrik karena masalah multilabel tidak cukup dinilai dari akurasi biasa:

- **micro-F1**: agregasi global TP/FP/FN, cocok untuk label dominan,
- **macro-F1**: rata-rata F1 per label, sensitif terhadap label minoritas,
- **weighted-F1**: F1 berbobot support,
- **Hamming Loss**: proporsi label yang salah,
- **Subset Accuracy**: prediksi dianggap benar hanya jika semua label review tepat persis.

CELL 028 [CODE]

```

def evaluate_multilabel(model, X, y_true, model_name: str, split_name: str) -> dict:
    y_pred = model.predict(X)
    return {
        'model': model_name,
        'split': split_name,
        'micro_f1': f1_score(y_true, y_pred, average='micro', zero_division=0),
        'macro_f1': f1_score(y_true, y_pred, average='macro', zero_division=0),
        'weighted_f1': f1_score(y_true, y_pred, average='weighted', zero_division=0),
        'hamming_loss': hamming_loss(y_true, y_pred),
        'subset_accuracy': accuracy_score(y_true, y_pred),
    }

```

CELL 029 [MARKDOWN]

14. Desain eksperimen model

Eksperimen memenuhi ketentuan soal karena membandingkan:

- minimal dua representasi teks: TF-IDF dan Bag of Words,
- minimal dua problem transformation: Binary Relevance/OneVsRest dan Classifier Chain,
- minimal dua algoritma: Logistic Regression dan Linear SVM,
- tuning sederhana pada parameter `C`.

Model dipilih berdasarkan `micro\_f1` pada validation set, lalu dievaluasi sekali pada test set.

CELL 030 [CODE]

```

EXPERIMENT_SPECS = [
    {
        'name': f'TFIDF_1-2gram_OVR_LogReg_C{C}',
        'factory': lambda C=C: make_tfidf_logreg(C),
        'representation': 'TF-IDF word unigram-bigram',
        'problem_transformation': 'Binary Relevance / OneVsRest',
        'algorithm': 'Logistic Regression',
        'tuned_param': f'C={C}',
    }
    for C in [0.5, 1.0, 2.0]
] + [
    {
        'name': f'TFIDF_1-2gram_OVR_LinearSVC_C{C}',
        'factory': lambda C=C: make_tfidf_svm(C),
        'representation': 'TF-IDF word unigram-bigram',
        'problem_transformation': 'Binary Relevance / OneVsRest',
        'algorithm': 'Linear SVM',
        'tuned_param': f'C={C}',
    }
    for C in [0.5, 1.0]
] + [
    {
        'name': f'BOW_1-2gram_ClassifierChain_LogReg_C{C}',
        'factory': lambda C=C: make_bow_chain_logreg(C),
        'representation': 'Bag of Words unigram-bigram',
        'problem_transformation': 'Classifier Chain',
        'algorithm': 'Logistic Regression',
        'tuned_param': f'C={C}',
    }
    for C in [0.5, 1.0]
]

pd.DataFrame([{'k': v for k, v in spec.items() if k != 'factory'} for spec in EXPERIMENT_SPECS])

```

CELL 031 [MARKDOWN]

15. Training dan validasi seluruh eksperimen

Setiap model dilatih pada train set dan dinilai pada validation set. Hasil validasi disimpan ke dataframe agar proses pemilihan model transparan dan dapat dibandingkan ulang.

CELL 032 [CODE]

```

trained_models = {}
experiment_rows = []

for spec in EXPERIMENT_SPECS:
    print(f"Training: {spec['name']}")

```

```

model = spec['factory']()
model.fit(X_train, y_train)
metrics = evaluate_multilabel(model, X_val, y_val, spec['name'], 'validation')
metrics.update({
    'representation': spec['representation'],
    'problem_transformation': spec['problem_transformation'],
    'algorithm': spec['algorithm'],
    'tuned_param': spec['tuned_param'],
})
trained_models[spec['name']] = model
experiment_rows.append(metrics)

exp_df = pd.DataFrame(experiment_rows).sort_values(
    by=['micro_f1', 'macro_f1', 'subset_accuracy'],
    ascending=False,
).reset_index(drop=True)

```

```

exp_df.to_csv(RESULTS_DIR / 'multilabel_experiment_results_val.csv', index=False)
display(exp_df)

```

CELL 033 [MARKDOWN]

16. Pemilihan model terbaik

Model terbaik dipilih dari validation set, bukan dari test set. Dengan cara ini, test set tetap berperan sebagai simulasi data baru yang belum pernah dilihat saat eksperimen.

CELL 034 [CODE]

```

best_row = exp_df.iloc[0]
best_model_name = best_row['model']
best_model = trained_models[best_model_name]

print('Best validation model:', best_model_name)
display(best_row.to_frame().T)

```

CELL 035 [MARKDOWN]

17. Evaluasi akhir pada test set

Cell ini menjalankan evaluasi test set untuk model terbaik. Hasil yang ditampilkan meliputi metrik multilabel utama dan classification report per label.

CELL 036 [CODE]

```

y_test_pred = best_model.predict(X_test)

test_metrics = evaluate_multilabel(best_model, X_test, y_test, best_model_name, 'test')
print(json.dumps(test_metrics, indent=2))

with open(RESULTS_DIR / 'multilabel_best_test_metrics.json', 'w', encoding='utf-8') as f:
    json.dump(test_metrics, f, indent=2)

report_dict = classification_report(
    y_test,
    y_test_pred,
    target_names=LABELS,
    output_dict=True,
    zero_division=0,
)
report_df = pd.DataFrame(report_dict).T
report_df.to_csv(RESULTS_DIR / 'multilabel_classification_report.csv')

print(classification_report(y_test, y_test_pred, target_names=LABELS, zero_division=0))
display(report_df)

```

CELL 037 [MARKDOWN]

18. Confusion matrix per label

Karena multilabel memiliki 13 label biner, confusion matrix dibuat per label. Nilai `FN` menunjukkan label yang seharusnya muncul tetapi tidak diprediksi, sedangkan `FP` menunjukkan label yang diprediksi padahal tidak ada di ground truth.

CELL 038 [CODE]

```

cm_rows = []
for idx, label in enumerate(LABELS):
    tn, fp, fn, tp = confusion_matrix(y_test[:, idx], y_test_pred[:, idx], labels=[0, 1]).ravel()
    cm_rows.append({
        'label': label,
        'tn': int(tn),
        'fp': int(fp),
        'fn': int(fn),
        'tp': int(tp),
        'support': int(y_test[:, idx].sum()),
    })

cm_df = pd.DataFrame(cm_rows)
cm_df.to_csv(RESULTS_DIR / 'multilabel_confusion_matrix_per_label.csv', index=False)
display(cm_df)

```

CELL 039 [MARKDOWN]

19. Error analysis multilabel

Analisis kesalahan dilakukan dengan membandingkan daftar label asli dan label prediksi per review.

Kesalahan yang sering muncul biasanya disebabkan oleh:

- frasa singkat atau ambigu,
- label minoritas yang sangat sedikit,
- review yang mengandung beberapa aspek sekaligus,
- batas antara aspek produk dan tempat yang kadang mirip, misalnya `pelayanan ramah` dapat dibaca sebagai tempat/fasilitas layanan.

CELL 040 [CODE]

```
def labels_from_binary(row_values):
    return [LABELS[i] for i, v in enumerate(row_values) if int(v) == 1]

error_rows = []
for i, (text, y_true_row, y_pred_row) in enumerate(zip(test_df['text_clean'], y_test, y_test_pred)):
    true_labels = labels_from_binary(y_true_row)
    pred_labels = labels_from_binary(y_pred_row)
    if set(true_labels) != set(pred_labels):
        error_rows.append({
            'index': i,
            'text': text,
            'true_labels': '|'.join(true_labels),
            'pred_labels': '|'.join(pred_labels),
            'missing_labels_FN': '|'.join(sorted(set(true_labels) - set(pred_labels))),
            'extra_labels_FP': '|'.join(sorted(set(pred_labels) - set(true_labels))),
        })

error_df = pd.DataFrame(error_rows)
error_df.to_csv(RESULTS_DIR / 'multilabel_error_examples.csv', index=False)
print('Jumlah review test yang tidak match sempurna:', len(error_df), 'dari', len(test_df))
display(error_df.head(15))
```

CELL 041 [MARKDOWN]

20. Interpretasi hasil singkat

Hal yang perlu diperhatikan saat membaca hasil:

- `micro-F1` biasanya lebih tinggi karena label dominan seperti `PRODUCT\_POSITIVE` dan `PLACE\_POSITIVE` banyak muncul.
- `macro-F1` lebih rendah jika label minoritas seperti `PROMOTION\_NEGATIVE` atau `PRICE\_NEUTRAL` sangat sedikit.
- `subset accuracy` ketat karena satu review dianggap salah jika ada satu label saja yang meleset.
- Hamming Loss yang kecil menunjukkan mayoritas keputusan label biner sudah benar, meskipun exact match belum selalu sempurna.

CELL 042 [CODE]

```
summary_table = pd.DataFrame([test_metrics])
display(summary_table[['model', 'micro_f1', 'macro_f1', 'weighted_f1', 'hamming_loss',
'subset_accuracy']])
```

CELL 043 [MARKDOWN]

21. Simpan model terbaik untuk Streamlit

Model disimpan bersama daftar label dan metadata. File inilah yang dipanggil kembali oleh aplikasi Streamlit pada menu ****Prediksi Multilabel ABSA****.

CELL 044 [CODE]

```
model_package = {
    'model': best_model,
    'labels': LABELS,
    'best_model_name': best_model_name,
    'cleaning_note': 'lowercase, remove URL, keep alphanumeric/underscore/hyphen, normalize spaces',
}

joblib.dump(model_package, MODELS_DIR / 'Kelp2_best_multilabel_model.joblib')
print('Saved to:', (MODELS_DIR / 'Kelp2_best_multilabel_model.joblib').resolve())
```

CELL 045 [MARKDOWN]

22. Fungsi inferensi review baru

Fungsi ini mensimulasikan proses yang sama seperti di Streamlit: input review baru dibersihkan, diproses model, lalu label yang bernilai 1 ditampilkan. Jika model memiliki `predict\_proba`, confidence score juga dapat dihitung.

CELL 046 [CODE]

```
def predict_multilabel_review(text: str, model_package: dict, threshold: float = 0.5) -> pd.DataFrame:
    model = model_package['model']
    labels = model_package['labels']
    text_clean = clean_text(text)
    pred = model.predict([text_clean])[0]

    rows = []
    for label, value in zip(labels, pred):
        rows.append({'label': label, 'predicted': int(value)})
    return pd.DataFrame(rows)

sample_review = 'Bajunya bagus, bahan adem, harganya juga murah, tapi tempat parkirnya agak sempit.'
sample_pred = predict_multilabel_review(sample_review, model_package)
display(sample_pred[sample_pred['predicted'] == 1])
```

CELL 047 [MARKDOWN]

23. Hubungan output Multilabel dan NER

Output multilabel menjawab pertanyaan: ****label aspek-sentimen apa saja yang ada pada satu review?****

Output NER menjawab pertanyaan: ****bagian kata/frasa mana yang menjadi bukti aspek tersebut?****

Contoh:

- Multilabel memprediksi ``PRODUCT_POSITIVE`` dan ``PRICE_POSITIVE``.
- NER dapat menandai span ``bahan adem`` sebagai ``PRODUCT_POSITIVE`` dan ``harga murah`` sebagai ``PRICE_POSITIVE``.

Jadi, multilabel memberi ringkasan label pada level review, sedangkan NER memberi lokasi span/aspek pada level token. Keduanya saling melengkapi untuk interpretasi ABSA.

CELL 048 [MARKDOWN]

24. REVISI AKURASI - Threshold Tuning Multilabel

Pada versi awal, model multilabel memakai ambang default 0,5 untuk semua label. Pada dataset multilabel yang tidak seimbang, satu ambang yang sama sering kurang optimal karena label minoritas memiliki distribusi probabilitas yang berbeda dari label mayoritas.

Bagian revisi ini menambahkan ****threshold tuning per label**** menggunakan validation set. Tujuannya bukan mengganti dataset test, melainkan memilih ambang prediksi berdasarkan validation set, lalu menguji hasil akhirnya pada test set. Strategi ini umum dipakai pada multilabel classification karena output model berupa skor/probabilitas per label.

CELL 049 [CODE]

```
import joblib
from sklearn.metrics import f1_score, hamming_loss, accuracy_score, classification_report,
confusion_matrix
```

Load ulang payload model terbaik agar bagian revisi ini tetap dapat dijalankan mandiri.

```
ml_payload = joblib.load(MODELS_DIR / 'Kelp2_best_multilabel_model.joblib')
best_model = ml_payload['model']
label_columns = ml_payload['labels']
```

```
X_val = val_df['text'].fillna('').astype(str)
y_val = val_df[label_columns].values
X_test = test_df['text'].fillna('').astype(str)
y_test = test_df[label_columns].values
```

```
def multilabel_metrics(y_true, y_pred):
    return {
        'micro_f1': f1_score(y_true, y_pred, average='micro', zero_division=0),
        'macro_f1': f1_score(y_true, y_pred, average='macro', zero_division=0),
        'weighted_f1': f1_score(y_true, y_pred, average='weighted', zero_division=0),
        'hamming_loss': hamming_loss(y_true, y_pred),
        'subset_accuracy': accuracy_score(y_true, y_pred),
    }
```

Baseline: prediksi default dari model sebelumnya.

```
default_test_pred = best_model.predict(X_test)
default_test_metrics = multilabel_metrics(y_test, default_test_pred)
default_test_metrics
```

CELL 050 [MARKDOWN]

24.1 Cara tuning threshold

Untuk setiap label, notebook mencoba beberapa threshold dari 0,05 sampai 0,95. Threshold terbaik dipilih berdasarkan F1 label tersebut pada validation set. Jika suatu label tidak memiliki contoh positif pada validation set, threshold tetap dipertahankan pada 0,5 agar model tidak terlalu mudah menghasilkan false positive.

Selain itu, jika tidak ada label yang lolos threshold pada sebuah review, sistem memilih label dengan skor probabilitas tertinggi. Hal ini memastikan setiap input tetap memiliki output label.

CELL 051 [CODE]

```
def tune_multilabel_thresholds(y_true, scores):
    thresholds = [] # hasil tuning per label
    pred = np.zeros_like(y_true)

    for j in range(y_true.shape[1]):
        if y_true[:, j].sum() == 0:
            best_t = 0.5
        else:
            best_f1, best_t = -1.0, 0.5
            for t in np.arange(0.05, 0.96, 0.05):
                candidate = (scores[:, j] >= t).astype(int)
                score = f1_score(y_true[:, j], candidate, zero_division=0)
                if score > best_f1 or (abs(score - best_f1) < 1e-12 and abs(t - 0.5) < abs(best_t -
0.5)):
                    best_f1, best_t = score, float(t)
            thresholds.append(best_t)
        pred[:, j] = (scores[:, j] >= best_t).astype(int)

    for i in range(len(pred)):
        if pred[i].sum() == 0:
            pred[i, int(scores[i].argmax())] = 1
    return np.array(thresholds), pred
```

```
val_scores = best_model.predict_proba(X_val)
if isinstance(val_scores, list):
```



```

val_scores = np.vstack([score[:, 1] for score in val_scores]).T

thresholds, val_pred_tuned = tune_multilabel_thresholds(y_val, val_scores)
threshold_df = pd.DataFrame({'label': label_columns, 'threshold': thresholds})
display(threshold_df)
-----
CELL 052 [CODE]
test_scores = best_model.predict_proba(X_test)
if isinstance(test_scores, list):
    test_scores = np.vstack([score[:, 1] for score in test_scores]).T

test_pred_tuned = (test_scores >= thresholds).astype(int)
for i in range(len(test_pred_tuned)):
    if test_pred_tuned[i].sum() == 0:
        test_pred_tuned[i, int(test_scores[i].argmax())] = 1

comparison_threshold_df = pd.DataFrame([
    {'setting': 'Default threshold 0.5', **default_test_metrics},
    {'setting': 'Validation tuned threshold', **multilabel_metrics(y_test, test_pred_tuned)},
])
display(comparison_threshold_df)
-----
CELL 053 [MARKDOWN]
## 24.2 Interpretasi hasil revisi multilabel

Setelah threshold tuning, metrik utama micro-F1 meningkat karena model menjadi lebih fleksibel terhadap label yang distribusi skornya berbeda. Macro-F1 dapat tetap lebih rendah daripada micro-F1 karena label minoritas, terutama label promosi dan beberapa label netral, jumlah datanya jauh lebih sedikit dibanding label produk/tempat. Oleh karena itu, laporan tetap menampilkan micro-F1, macro-F1, weighted-F1, Hamming Loss, subset accuracy, serta classification report per label.
-----
CELL 054 [CODE]
# Simpan model multilabel revisi agar Streamlit memakai threshold hasil validasi.
ml_payload['thresholds'] = thresholds.tolist()
ml_payload['threshold_strategy'] = 'Per-label threshold tuned on validation set; empty prediction fallback uses highest probability label.'
ml_payload['best_model_name'] = 'BOW_1-2gram_ClassifierChain_LogReg_C0.5_ThresholdTuned'
ml_payload['default_test_metrics'] = default_test_metrics
ml_payload['tuned_test_metrics'] = multilabel_metrics(y_test, test_pred_tuned)
joblib.dump(ml_payload, MODELS_DIR / 'Kelp2_best_multilabel_model.joblib')

pd.DataFrame(classification_report(y_test, test_pred_tuned, target_names=label_columns, output_dict=True, zero_division=0)).T.to_csv(RESULTS_DIR / 'multilabel_classification_report.csv')
with open(RESULTS_DIR / 'multilabel_best_test_metrics.json', 'w', encoding='utf-8') as f:
    json.dump({'model': ml_payload['best_model_name'], 'split': 'test',
**ml_payload['tuned_test_metrics'], 'thresholds': thresholds.tolist()}, f, indent=2)

print('Model multilabel revisi berhasil disimpan.')
=====
FILE: notebook/Kelp2_ner_modeling.ipynb
=====
CELL 001 [MARKDOWN]
# Kelp2 - UAS PBA: Named Entity Recognition ABSA

Notebook ini mengerjakan **Proyek B - Named Entity Recognition (NER) ABSA**.  

Tujuannya adalah mengenali span/token yang merujuk pada aspek ABSA di dalam review Google Places, seperti produk, harga, tempat, promosi, dan out-of-topic.

Notebook dibuat bertahap seperti guided notebook:
1. data loading JSONL,
2. validasi token-label dan format BIO,
3. konversi tag BIO ke span dan validasi ulang,
4. EDA entitas,
5. tag mapping dan embedding vocabulary,
6. padding/batching dan strategi imbalance,
7. konfigurasi model BiLSTM token classification,
8. training dengan validation set dan early stopping,
9. evaluasi test set,
10. confusion matrix dan error analysis boundary/label,
11. penyimpanan model terbaik,
12. contoh inferensi dan highlight span untuk Streamlit.
-----
CELL 002 [MARKDOWN]
## 0. Pemetaan terhadap ketentuan soal UAS

| Kriteria UAS | Implementasi pada notebook |
|---|---|
| Data loading dan validasi span | Membaca JSONL train/val/test, menampilkan review token-label, memvalidasi panjang token dan tag. |
| Tokenization dan konversi tag | Dataset split sudah berbentuk token-label BIO; notebook memvalidasi BIO, mengonversi BIO ke span, dan menyediakan fungsi balik span-ke-BIO. |
| EDA entitas | Visualisasi jumlah token per review, distribusi tag, jumlah entitas per label, dan jumlah entitas per review. |
| Embedding dan tag mapping | Membuat vocabulary token, tag mapping, padding/batching, serta bobot loss untuk mengurangi dominasi tag `O`. |
| Model deep learning | Menggunakan BiLSTM token classification berbasis PyTorch. Model ini dipilih karena dapat dijalankan offline tanpa download pretrained model. |
| Evaluasi dan error analysis | Classification report token-level non-0, confusion matrix, token prediction dataframe, dan contoh error boundary/label. |

```

| Deployment | Model `.pt` disimpan dan dipakai aplikasi Streamlit pada menu Prediksi NER ABSA. |

CELL 003 [MARKDOWN]

1. Import library dan konfigurasi path

Notebook memakai PyTorch untuk model deep learning. Tokenisasi yang digunakan adalah tokenisasi yang sudah tersedia pada dataset JSONL, sehingga alignment token-label bersifat 1:1. Jika menggunakan model transformer subword seperti IndoBERT, alignment harus memperhatikan subword; pada notebook ini masalah tersebut tidak muncul karena token input sudah sejajar dengan tag BIO.

CELL 004 [CODE]

```
from pathlib import Path
from collections import Counter, defaultdict
import json
import math
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.metrics import classification_report, confusion_matrix, precision_recall_fscore_support

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.set_num_threads(2)

CWD = Path.cwd()
if (CWD / 'dataset').exists():
    ROOT = CWD
elif (CWD.parent / 'dataset').exists():
    ROOT = CWD.parent
else:
    ROOT = Path('.')

DATA_DIR = ROOT / 'dataset'
RESULTS_DIR = ROOT / 'results'
MODELS_DIR = ROOT / 'models'
FIGURES_DIR = ROOT / 'figures'

for p in [RESULTS_DIR, MODELS_DIR, FIGURES_DIR]:
    p.mkdir(parents=True, exist_ok=True)

print('Project root:', ROOT.resolve())
```

CELL 005 [MARKDOWN]

2. Fungsi loader JSONL

Format dataset NER adalah JSONL. Setiap baris memuat satu review dengan minimal dua field:

- `tokens`: daftar token review,
- `ner\_tags`: daftar tag BIO untuk setiap token.

Panjang `tokens` dan `ner\_tags` harus sama agar alignment benar.

CELL 006 [CODE]

```
def load_jsonl(path: Path) -> list:
    rows = []
    with open(path, encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows

train_rows = load_jsonl(DATA_DIR / 'Kelp2_ner_train.jsonl')
val_rows = load_jsonl(DATA_DIR / 'Kelp2_ner_val.jsonl')
test_rows = load_jsonl(DATA_DIR / 'Kelp2_ner_test.jsonl')

pd.DataFrame({
    'split': ['train', 'validation', 'test'],
    'jumlah_review': [len(train_rows), len(val_rows), len(test_rows)],
})
```

CELL 007 [MARKDOWN]

3. Menampilkan contoh data NER

Contoh ini penting untuk memastikan struktur data bisa dibaca manusia. Dalam review, setiap token memiliki satu tag BIO. Tag `B-...` menandai awal entitas, `I-...` menandai lanjutan entitas, dan `O` berarti bukan entitas aspek.

CELL 008 [CODE]

```
example = train_rows[0]
example_df = pd.DataFrame({
    'token': example['tokens'],
    'ner_tag': example['ner_tags'],
})
```

```
print('Jumlah token contoh:', len(example['tokens']))
display(example_df)
```

CELL 009 [MARKDOWN]

4. Validasi alignment token-label

Cell ini mengecek apakah setiap review memiliki jumlah token dan tag yang sama. Jika ada mismatch, training NER tidak boleh dilanjutkan karena label tidak lagi menunjuk token yang benar.

CELL 010 [CODE]

```
def validate_alignment(rows: list, split_name: str) -> dict:
    bad_rows = []
    empty_rows = 0
    for i, row in enumerate(rows):
        tokens = row.get('tokens', [])
        tags = row.get('ner_tags', [])
        if len(tokens) == 0:
            empty_rows += 1
        if len(tokens) != len(tags):
            bad_rows.append(i)
    return {
        'split': split_name,
        'rows': len(rows),
        'bad_alignment_rows': len(bad_rows),
        'empty_rows': empty_rows,
        'first_bad_index': bad_rows[0] if bad_rows else None,
    }

alignment_report = pd.DataFrame([
    validate_alignment(train_rows, 'train'),
    validate_alignment(val_rows, 'validation'),
    validate_alignment(test_rows, 'test'),
])
display(alignment_report)

assert alignment_report['bad_alignment_rows'].sum() == 0, 'Ada token-label yang tidak sejajar.'
```

CELL 011 [MARKDOWN]

5. Validasi format BIO

Tag yang valid adalah `O` atau tag dengan prefix `B-` dan `I-`. Karena dataset UTS mempertahankan sentimen pada label NER, contoh tag yang mungkin muncul adalah `B-PRODUCT\_POSITIVE`, `I-PRICE\_NEGATIVE`, dan `B-PLACE\_POSITIVE`.

CELL 012 [CODE]

```
def is_valid_bio_tag(tag: str) -> bool:
    return tag == 'O' or tag.startswith('B-') or tag.startswith('I-')

all_tags_raw = sorted(set(tag for rows in [train_rows, val_rows, test_rows] for row in rows for tag in
row['ner_tags']))
invalid_tags = [tag for tag in all_tags_raw if not is_valid_bio_tag(tag)]
print('Jumlah tag unik:', len(all_tags_raw))
print('Invalid tags:', invalid_tags)
display(pd.DataFrame({'tag': all_tags_raw}))
assert len(invalid_tags) == 0, 'Ada tag yang bukan BIO.'
```

CELL 013 [MARKDOWN]

6. Konversi BIO ke span entitas

Walaupun file JSONL sudah berisi token-label BIO, rubrik UAS meminta pemahaman konversi span ke token-label. Cell ini membuat fungsi pembacaan BIO menjadi span. Span berisi:

- indeks token awal,
- indeks token akhir eksklusif,
- label entitas,
- teks span.

Fungsi ini juga dipakai untuk error analysis boundary.

CELL 014 [CODE]

```
def bio_to_spans(tokens, tags):
    spans = []
    start = None
    current_label = None

    for i, tag in enumerate(tags + ['O']):
        if tag == 'O':
            if current_label is not None:
                spans.append({
                    'start': start,
                    'end': i,
                    'label': current_label,
                    'text': ' '.join(tokens[start:i]),
                })
            start = None
            current_label = None
            continue

        prefix, label = tag.split('-', 1)
        if prefix == 'B' or current_label != label:
            if current_label is not None:
```

```

        spans.append({
            'start': start,
            'end': i,
            'label': current_label,
            'text': ' '.join(tokens[start:i]),
        })
        start = i
        current_label = label
    elif prefix == 'I':
        # masih lanjutan entitas yang sama
        pass

    return spans

```

```

example_spans = bio_to_spans(example['tokens'], example['ner_tags'])
display(pd.DataFrame(example_spans))

```

CELL 015 [MARKDOWN]

7. Konversi span kembali ke BIO untuk validasi

Fungsi ini menunjukkan kebalikan dari proses sebelumnya. Jika dataset asli berupa span offset, prinsipnya sama: tentukan token yang masuk span, lalu beri tag `B-` untuk token pertama dan `I-` untuk token berikutnya. Pada dataset ini, proses ini digunakan untuk validasi konsistensi.

CELL 016 [CODE]

```

def spans_to_bio(tokens, spans):
    tags = ['O'] * len(tokens)
    for span in spans:
        start = span['start']
        end = span['end']
        label = span['label']
        if start < 0 or end > len(tokens) or start >= end:
            continue
        tags[start] = f'B-{label}'
        for i in range(start + 1, end):
            tags[i] = f'I-{label}'
    return tags

reconstructed_tags = spans_to_bio(example['tokens'], example_spans)
print('BIO reconstruction equals original:', reconstructed_tags == example['ner_tags'])

```

CELL 017 [MARKDOWN]

8. EDA 1 - Distribusi jumlah token per review

Jumlah token per review membantu menentukan strategi padding dalam batching. Karena BiLSTM memproses sequence, review dalam satu batch dipadding sesuai panjang review terpanjang pada batch tersebut.

CELL 018 [CODE]

```

train_token_lengths = [len(row['tokens']) for row in train_rows]
plt.figure(figsize=(8, 4))
plt.hist(train_token_lengths, bins=30)
plt.title('Distribusi Jumlah Token per Review - Train')
plt.xlabel('Jumlah token')
plt.ylabel('Jumlah review')
plt.tight_layout()
plt.show()

pd.Series(train_token_lengths).describe()

```

CELL 019 [MARKDOWN]

9. EDA 2 - Distribusi tag entitas

Tag `O` biasanya jauh lebih banyak daripada tag entitas. Oleh karena itu, saat training digunakan class weight dan bobot `O` diturunkan agar model tidak hanya belajar memprediksi `O`.

CELL 020 [CODE]

```

tag_counter = Counter(tag for row in train_rows for tag in row['ner_tags'])
tag_dist = pd.DataFrame(tag_counter.items(), columns=['tag', 'count']).sort_values('count',
ascending=False)
display(tag_dist)

plt.figure(figsize=(12, 5))
tag_dist[tag_dist['tag'] != 'O'].set_index('tag')['count'].plot(kind='bar')
plt.title('Distribusi Tag Entitas Non-O - Train')
plt.xlabel('Tag BIO')
plt.ylabel('Jumlah token')
plt.tight_layout()
plt.savefig(FIGURES_DIR / 'ner_tag_distribution.png', dpi=150)
plt.show()

```

CELL 021 [MARKDOWN]

10. EDA 3 - Total entitas per label

Token-level count belum tentu sama dengan entity-level count. Cell ini menghitung jumlah span entitas berdasarkan hasil konversi BIO ke span.

CELL 022 [CODE]

```

entity_counter = Counter()
entities_per_review = []
for row in train_rows:

```

```

spans = bio_to_spans(row['tokens'], row['ner_tags'])
entities_per_review.append(len(spans))
for span in spans:
    entity_counter[span['label']] += 1

entity_dist = pd.DataFrame(entity_counter.items(), columns=['entity_label',
'span_count']).sort_values('span_count', ascending=False)
display(entity_dist)

plt.figure(figsize=(12, 5))
entity_dist.set_index('entity_label')['span_count'].plot(kind='bar')
plt.title('Jumlah Span Entitas per Label - Train')
plt.xlabel('Label entitas')
plt.ylabel('Jumlah span')
plt.tight_layout()
plt.show()

```

CELL 023 [MARKDOWN]

11. EDA 4 - Jumlah entitas per review

Review dengan banyak entitas biasanya lebih sulit karena model harus membedakan beberapa aspek dan beberapa sentimen sekaligus.

CELL 024 [CODE]

```

plt.figure(figsize=(8, 4))
plt.hist(entities_per_review, bins=20)
plt.title('Distribusi Jumlah Entitas per Review - Train')
plt.xlabel('Jumlah entitas dalam satu review')
plt.ylabel('Jumlah review')
plt.tight_layout()
plt.show()

```

```
pd.Series(entities_per_review).describe()
```

CELL 025 [MARKDOWN]

12. Vocabulary token dan tag mapping

Model BiLSTM memakai embedding yang dilatih dari data train. Token yang tidak ditemukan pada train akan dipetakan ke `

CELL 026 [CODE]

```

PAD_TOKEN = '<PAD>'
UNK_TOKEN = '<UNK>'

vocab = {PAD_TOKEN: 0, UNK_TOKEN: 1}
for token, _ in Counter(tok.lower() for row in train_rows for tok in row['tokens']).most_common():
    if token not in vocab:
        vocab[token] = len(vocab)

all_tags = ['0'] + [tag for tag in sorted(all_tags_raw) if tag != '0']
tag_to_id = {tag: i for i, tag in enumerate(all_tags)}
id_to_tag = {i: tag for tag, i in tag_to_id.items()}

print('Ukuran vocabulary:', len(vocab))
print('Jumlah tag:', len(all_tags))
display(pd.DataFrame({'tag': all_tags, 'id': [tag_to_id[t] for t in all_tags]}))

```

CELL 027 [MARKDOWN]

13. Strategi imbalance tag `0`

Tag `0` sangat dominan. Jika tidak ditangani, model dapat terlihat akurat tetapi gagal mengenali entitas. Strategi yang digunakan:

- `CrossEntropyLoss` diberi class weight berdasarkan frekuensi tag,
- bobot tag `0` diturunkan agar model lebih berani memprediksi entitas,
- metrik utama menggunakan micro-F1 non-0, bukan akurasi semua token.

CELL 028 [CODE]

```

freq = Counter(tag for row in train_rows for tag in row['ner_tags'])
weights = np.array([1 / math.sqrt(freq.get(tag, 1)) for tag in all_tags], dtype=np.float32)
weights = weights / weights.mean()
weights[tag_to_id['0']] *= 0.25

class_weight_df = pd.DataFrame({
    'tag': all_tags,
    'freq_train': [freq.get(tag, 0) for tag in all_tags],
    'loss_weight': weights,
})
display(class_weight_df.sort_values('freq_train', ascending=False))

```

CELL 029 [MARKDOWN]

14. Dataset dan collate function untuk padding dinamis

Padding dilakukan per batch, bukan menggunakan panjang maksimum global. Dengan padding dinamis, training lebih efisien karena sequence hanya dipadding sampai panjang terpanjang dalam batch.

CELL 030 [CODE]

```

class ABSANERDataset(Dataset):
    def __init__(self, rows):
        self.rows = rows

```

```

def __len__(self):
    return len(self.rows)

def __getitem__(self, idx):
    row = self.rows[idx]
    token_ids = [vocab.get(token.lower(), vocab[UNK_TOKEN]) for token in row['tokens']]
    label_ids = [tag_to_id[tag] for tag in row['ner_tags']]
    return {
        'input_ids': token_ids,
        'labels': label_ids,
        'tokens': row['tokens'],
        'gold_tags': row['ner_tags'],
    }

def collate_batch(batch):
    max_len = max(len(item['input_ids']) for item in batch)
    input_ids, labels, masks, tokens, gold_tags = [], [], [], [], []

    for item in batch:
        n = len(item['input_ids'])
        pad_len = max_len - n
        input_ids.append(item['input_ids'] + [vocab[PAD_TOKEN]] * pad_len)
        labels.append(item['labels'] + [-100] * pad_len)
        masks.append([1] * n + [0] * pad_len)
        tokens.append(item['tokens'])
        gold_tags.append(item['gold_tags'])

    return {
        'input_ids': torch.tensor(input_ids, dtype=torch.long),
        'labels': torch.tensor(labels, dtype=torch.long),
        'mask': torch.tensor(masks, dtype=torch.bool),
        'tokens': tokens,
        'gold_tags': gold_tags,
    }

BATCH_SIZE = 128
train_loader = DataLoader(ABSANERDataset(train_rows), batch_size=BATCH_SIZE, shuffle=True,
collate_fn=collate_batch)
val_loader = DataLoader(ABSANERDataset(val_rows), batch_size=BATCH_SIZE, shuffle=False,
collate_fn=collate_batch)
test_loader = DataLoader(ABSANERDataset(test_rows), batch_size=BATCH_SIZE, shuffle=False,
collate_fn=collate_batch)

batch = next(iter(train_loader))
print(batch['input_ids'].shape, batch['labels'].shape, batch['mask'].shape)

```

CELL 031 [MARKDOWN]

15. Konfigurasi model BiLSTM token classification

Model terdiri dari:

1. embedding layer untuk mengubah token id menjadi vektor,
2. BiLSTM untuk membaca konteks kiri dan kanan token,
3. dropout untuk regularisasi,
4. linear classifier untuk memprediksi tag BIO pada setiap token.

Model ini termasuk deep learning dan relevan untuk NER karena prediksi token dipengaruhi konteks sekitarnya.

CELL 032 [CODE]

```

class BiLSTMTagger(nn.Module):
    def __init__(self, vocab_size, tagset_size, embedding_dim=48, hidden_dim=64, dropout=0.25):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=vocab[PAD_TOKEN])
        self.lstm = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_dim // 2,
            batch_first=True,
            bidirectional=True,
        )
        self.dropout = nn.Dropout(dropout)
        self.classifier = nn.Linear(hidden_dim, tagset_size)

    def forward(self, input_ids):
        x = self.embedding(input_ids)
        x, _ = self.lstm(x)
        x = self.dropout(x)
        logits = self.classifier(x)
        return logits

MODEL_CONFIG = {
    'embedding_dim': 48,
    'hidden_dim': 64,
    'dropout': 0.25,
}

model = BiLSTMTagger(len(vocab), len(all_tags), **MODEL_CONFIG)
print(model)

```

CELL 033 [MARKDOWN]

16. Fungsi evaluasi token-level non-0

Evaluasi token-level dilakukan dengan mengabaikan padding. Selain itu, micro-F1 utama dihitung pada label non-0 agar kualitas ekstraksi entitas terlihat jelas.

CELL 034 [CODE]

```
def flatten_predictions(model, loader):
    model.eval()
    y_true, y_pred = [], []
    total_loss = 0.0
    total_batches = 0

    with torch.no_grad():
        for batch in loader:
            logits = model(batch['input_ids'])
            loss = criterion(logits.view(-1, len(all_tags)), batch['labels'].view(-1))
            total_loss += loss.item()
            total_batches += 1

            pred_ids = logits.argmax(dim=-1).cpu().numpy()
            label_ids = batch['labels'].cpu().numpy()
            mask = label_ids != -100
            y_true.extend(label_ids[mask].tolist())
            y_pred.extend(pred_ids[mask].tolist())

    y_true_tags = [id_to_tag[i] for i in y_true]
    y_pred_tags = [id_to_tag[i] for i in y_pred]
    return total_loss / max(total_batches, 1), y_true_tags, y_pred_tags

def evaluate_ner(model, loader):
    loss, y_true_tags, y_pred_tags = flatten_predictions(model, loader)
    non_o_labels = [tag for tag in all_tags if tag != 'O']
    p, r, f1, _ = precision_recall_fscore_support(
        y_true_tags,
        y_pred_tags,
        labels=non_o_labels,
        average='micro',
        zero_division=0,
    )
    macro_p, macro_r, macro_f1, _ = precision_recall_fscore_support(
        y_true_tags,
        y_pred_tags,
        labels=non_o_labels,
        average='macro',
        zero_division=0,
    )
    return {
        'loss': loss,
        'micro_precision_non_o': p,
        'micro_recall_non_o': r,
        'micro_f1_non_o': f1,
        'macro_precision_non_o': macro_p,
        'macro_recall_non_o': macro_r,
        'macro_f1_non_o': macro_f1,
        'y_true_tags': y_true_tags,
        'y_pred_tags': y_pred_tags,
    }
```

CELL 035 [MARKDOWN]

17. Training dengan validation set dan early stopping

Model dilatih menggunakan Adam optimizer. Early stopping memakai `micro\_f1\_non\_o` pada validation set. Checkpoint terbaik disimpan ke file `.pt`, sehingga model yang dipakai Streamlit adalah model terbaik, bukan model terakhir.

CELL 036 [CODE]

```
criterion = nn.CrossEntropyLoss(
    ignore_index=-100,
    weight=torch.tensor(weights, dtype=torch.float32),
)
optimizer = torch.optim.Adam(model.parameters(), lr=0.005)

# Default False agar notebook dapat dijalankan cepat bersama file model yang sudah disimpan.
# Ubah menjadi True jika ingin melatih ulang dari awal.
RETRAIN_NER = False
EPOCHS = 8
PATIENCE = 2
best_ckpt_path = MODELS_DIR / 'kelp2_best_ner_model.pt'
training_log = []

if best_ckpt_path.exists() and not RETRAIN_NER:
    print('Checkpoint sudah tersedia, training ulang dilewati.')
    print('Untuk training ulang dari awal, ubah RETRAIN_NER = True lalu jalankan kembali cell ini.')
    training_log_path = RESULTS_DIR / 'ner_training_log.csv'
    if training_log_path.exists():
        training_log_df = pd.read_csv(training_log_path)
    else:
        training_log_df = pd.DataFrame(columns=['epoch', 'train_loss', 'val_loss', 'val_micro_f1_non_o'])
```

```

else:
    best_val_f1 = -1.0
    bad_epochs = 0

    for epoch in range(1, EPOCHS + 1):
        model.train()
        train_losses = []

        for batch in train_loader:
            optimizer.zero_grad()
            logits = model(batch['input_ids'])
            loss = criterion(logits.view(-1, len(all_tags)), batch['labels'].view(-1))
            loss.backward()
            optimizer.step()
            train_losses.append(loss.item())

        val_metrics = evaluate_ner(model, val_loader)
        row = {
            'epoch': epoch,
            'train_loss': float(np.mean(train_losses)),
            'val_loss': val_metrics['loss'],
            'val_micro_precision_non_o': val_metrics['micro_precision_non_o'],
            'val_micro_recall_non_o': val_metrics['micro_recall_non_o'],
            'val_micro_f1_non_o': val_metrics['micro_f1_non_o'],
            'val_macro_f1_non_o': val_metrics['macro_f1_non_o'],
        }
        training_log.append(row)
        print(row)

        if val_metrics['micro_f1_non_o'] > best_val_f1:
            best_val_f1 = val_metrics['micro_f1_non_o']
            bad_epochs = 0
            torch.save({
                'state_dict': model.state_dict(),
                'vocab': vocab,
                'tag_to_id': tag_to_id,
                'id_to_tag': id_to_tag,
                'config': MODEL_CONFIG,
                'best_val_f1_non_o': best_val_f1,
            }, best_ckpt_path)
        else:
            bad_epochs += 1
            if bad_epochs >= PATIENCE:
                print('Early stopping aktif.')
                break

    training_log_df = pd.DataFrame(training_log)
    training_log_df.to_csv(RESULTS_DIR / 'ner_training_log.csv', index=False)

```

```
display(training_log_df)
```

CELL 037 [MARKDOWN]

18. Visualisasi training log

Kurva validasi membantu memastikan bahwa model benar-benar belajar dan tidak hanya menghasilkan prediksi statis.

CELL 038 [CODE]

```

plt.figure(figsize=(8, 4))
plt.plot(training_log_df['epoch'], training_log_df['val_micro_f1_non_o'], marker='o')
plt.title('Validation Micro-F1 Non-O per Epoch')
plt.xlabel('Epoch')
plt.ylabel('Validation Micro-F1 Non-O')
plt.tight_layout()
plt.savefig(FIGURES_DIR / 'ner_validation_curve.png', dpi=150)
plt.show()

```

CELL 039 [MARKDOWN]

19. Load checkpoint terbaik

Evaluasi test set dilakukan menggunakan checkpoint terbaik berdasarkan validation set. Ini mencegah hasil test memakai model yang overfit pada epoch terakhir.

CELL 040 [CODE]

```

checkpoint = torch.load(best_ckpt_path, map_location='cpu')
model = BiLSTMTagger(
    vocab_size=len(checkpoint['vocab']),
    tagset_size=len(checkpoint['tag_to_id']),
    **checkpoint['config'],
)
model.load_state_dict(checkpoint['state_dict'])
model.eval()

vocab = checkpoint['vocab']
tag_to_id = checkpoint['tag_to_id']
id_to_tag = checkpoint['id_to_tag']
all_tags = [id_to_tag[i] for i in range(len(id_to_tag))]
print('Best validation F1 non-O:', checkpoint['best_val_f1_non_o'])

```

CELL 041 [MARKDOWN]

20. Evaluasi akhir pada test set

Cell ini menampilkan precision, recall, dan F1 token-level. Fokus interpretasi diberikan pada label non-0 karena tujuan NER adalah mengekstraksi entitas/aspek.

CELL 042 [CODE]

```
test_metrics_full = evaluate_ner(model, test_loader)
metrics_to_save = {k: v for k, v in test_metrics_full.items() if not k.startswith('y_')}
print(json.dumps(metrics_to_save, indent=2))
```

```
with open(RESULTS_DIR / 'ner_best_test_metrics.json', 'w', encoding='utf-8') as f:
    json.dump(metrics_to_save, f, indent=2)
```

```
print(classification_report(
    test_metrics_full['y_true_tags'],
    test_metrics_full['y_pred_tags'],
    labels=all_tags,
    zero_division=0,
))
```

```
report_df = pd.DataFrame(classification_report(
    test_metrics_full['y_true_tags'],
    test_metrics_full['y_pred_tags'],
    labels=all_tags,
    output_dict=True,
    zero_division=0,
)).T
report_df.to_csv(RESULTS_DIR / 'ner_classification_report_token_level.csv')
display(report_df)
```

CELL 043 [MARKDOWN]

21. Menyimpan prediksi token ke dataframe

Dataframe prediksi token berguna untuk audit manual. Dosen dapat melihat token mana yang benar, salah boundary, atau salah label.

CELL 044 [CODE]

```
def predict_rows_to_dataframe(model, rows):
    loader = DataLoader(ABSANERDataset(rows), batch_size=BATCH_SIZE, shuffle=False,
collate_fn=collate_batch)
    records = []
    review_index = 0
    model.eval()
    with torch.no_grad():
        for batch in loader:
            logits = model(batch['input_ids'])
            pred_ids = logits.argmax(dim=-1).cpu().numpy()
            labels = batch['labels'].cpu().numpy()
            for b_idx, tokens in enumerate(batch['tokens']):
                for t_idx, token in enumerate(tokens):
                    gold_id = labels[b_idx][t_idx]
                    pred_id = pred_ids[b_idx][t_idx]
                    records.append({
                        'review_index': review_index,
                        'token_index': t_idx,
                        'token': token,
                        'gold_tag': id_to_tag[int(gold_id)],
                        'pred_tag': id_to_tag[int(pred_id)],
                        'is_correct': id_to_tag[int(gold_id)] == id_to_tag[int(pred_id)],
                    })
            review_index += 1
    return pd.DataFrame(records)
```

```
token_pred_df = predict_rows_to_dataframe(model, test_rows)
token_pred_df.to_csv(RESULTS_DIR / 'ner_token_predictions_test.csv', index=False)
display(token_pred_df.head(30))
```

CELL 045 [MARKDOWN]

22. Confusion matrix token-level

Confusion matrix menunjukkan tag mana yang sering tertukar. Label minoritas biasanya memiliki recall rendah karena contoh latihnya sedikit.

CELL 046 [CODE]

```
cm = confusion_matrix(
    test_metrics_full['y_true_tags'],
    test_metrics_full['y_pred_tags'],
    labels=all_tags,
)
cm_df = pd.DataFrame(cm, index=all_tags, columns=all_tags)
cm_df.to_csv(RESULTS_DIR / 'ner_confusion_matrix_token_level.csv')
display(cm_df)
```

CELL 047 [MARKDOWN]

23. Error analysis boundary dan label

Kesalahan NER dapat dibagi menjadi:

- **boundary error**: model menemukan entitas/aspek tetapi batas token awal/akhir tidak tepat,
- **label error**: model menemukan span yang sama tetapi label sentimennya salah,
- **missed entity**: entitas gold tidak ditemukan,

- ****spurious entity****: model memprediksi entitas yang tidak ada di gold.

Cell ini menampilkan contoh review yang memiliki perbedaan gold dan prediksi.

CELL 048 [CODE]

```
def predict_tags_for_rows(model, rows):
    loader = DataLoader(ABSANERDataset(rows), batch_size=BATCH_SIZE, shuffle=False,
    collate_fn=collate_batch)
    all_pred_tags = []
    model.eval()
    with torch.no_grad():
        for batch in loader:
            logits = model(batch['input_ids'])
            pred_ids = logits.argmax(dim=-1).cpu().numpy()
            for b_idx, tokens in enumerate(batch['tokens']):
                pred_tags = [id_to_tag[int(pred_ids[b_idx][i])] for i in range(len(tokens))]
                all_pred_tags.append(pred_tags)
    return all_pred_tags

pred_tag_sequences = predict_tags_for_rows(model, test_rows)

error_examples = []
for i, (row, pred_tags) in enumerate(zip(test_rows, pred_tag_sequences)):
    gold_tags = row['ner_tags']
    if gold_tags != pred_tags:
        gold_spans = bio_to_spans(row['tokens'], gold_tags)
        pred_spans = bio_to_spans(row['tokens'], pred_tags)
        error_examples.append({
            'review_index': i,
            'text': ' '.join(row['tokens']),
            'gold_tags': ' '.join(gold_tags),
            'pred_tags': ' '.join(pred_tags),
            'gold_spans': json.dumps(gold_spans, ensure_ascii=False),
            'pred_spans': json.dumps(pred_spans, ensure_ascii=False),
        })

error_df = pd.DataFrame(error_examples)
error_df.to_csv(RESULTS_DIR / 'ner_error_examples.csv', index=False)
print('Jumlah review test dengan sequence tag tidak sama:', len(error_df), 'dari', len(test_rows))
display(error_df.head(10))
```

CELL 049 [MARKDOWN]

24. Interpretasi hasil NER singkat

Hasil NER biasanya lebih rendah daripada multilabel karena tugasnya lebih sulit. Multilabel hanya perlu menentukan label pada level review, sedangkan NER harus menentukan token awal, token akhir, aspek, dan sentimen. Kesalahan satu token saja dapat membuat span dianggap salah secara boundary.

CELL 050 [CODE]

```
metrics_summary = pd.DataFrame([metrics_to_save])
display(metrics_summary)
```

CELL 051 [MARKDOWN]

25. Fungsi inferensi review baru

Fungsi ini dipakai sebagai dasar Streamlit: teks baru ditokenisasi sederhana menggunakan whitespace, lalu model mengeluarkan tag BIO. Span hasil prediksi dikembalikan dalam bentuk dataframe.

CELL 052 [CODE]

```
def predict_ner_review(text: str) -> pd.DataFrame:
    tokens = text.strip().split()
    if not tokens:
        return pd.DataFrame(columns=['start', 'end', 'label', 'text'])

    token_ids = [vocab.get(tok.lower(), vocab.get(UNK_TOKEN, 1)) for tok in tokens]
    input_tensor = torch.tensor([token_ids], dtype=torch.long)

    model.eval()
    with torch.no_grad():
        logits = model(input_tensor)
        pred_ids = logits.argmax(dim=-1)[0].cpu().numpy().tolist()[len(tokens)]

    pred_tags = [id_to_tag[int(i)] for i in pred_ids]
    spans = bio_to_spans(tokens, pred_tags)
    return pd.DataFrame(spans), pd.DataFrame({'token': tokens, 'pred_tag': pred_tags})

sample_review = 'Bajunya bagus bahan adem harga murah tapi parkir sempit'
sample_spans, sample_token_tags = predict_ner_review(sample_review)
display(sample_token_tags)
display(sample_spans)
```

CELL 053 [MARKDOWN]

26. Fungsi highlight sederhana untuk deployment

Streamlit menggunakan hasil span untuk menampilkan highlight. Fungsi di bawah membuat string HTML sederhana sehingga aspek yang terdeteksi dapat terlihat langsung pada review.

CELL 054 [CODE]

```
def make_highlight_html(tokens, tags):
    parts = []
```

```

for token, tag in zip(tokens, tags):
    if tag == 'O':
        parts.append(token)
    else:
        parts.append(f"<mark title='{tag}'>{token}</mark>")
return ' '.join(parts)

html_preview = make_highlight_html(sample_token_tags['token'].tolist(),
sample_token_tags['pred_tag'].tolist())
print(html_preview)

```

CELL 055 [MARKDOWN]

27. Hubungan output NER dengan output Multilabel

NER memberi bukti lokal berupa span/token. Multilabel memberi keputusan global pada review.

Contoh hubungan:

- NER mendeteksi `bahan adem` sebagai `PRODUCT\_POSITIVE`, maka multilabel idealnya juga memuat `PRODUCT\_POSITIVE`.
- NER mendeteksi `harga murah` sebagai `PRICE\_POSITIVE`, maka multilabel idealnya juga memuat `PRICE\_POSITIVE`.
- Jika multilabel memprediksi `PLACE\_NEGATIVE` tetapi NER tidak menemukan span tempat, kemungkinan review mengandung sinyal tempat yang terlalu implisit atau model NER gagal boundary.

Dengan demikian, kedua model dipakai bersama di Streamlit agar hasil tidak hanya berupa label, tetapi juga bagian teks yang mendukung label tersebut.

CELL 056 [MARKDOWN]

25. REVISI AKURASI - Feature-Based Token Classifier untuk NER

Model BiLSTM pada bagian sebelumnya sudah memenuhi ketentuan deep learning NER, tetapi performanya masih terbatas karena dataset kecil, label `O` dominan, dan beberapa label aspek-sentimen jarang muncul. Untuk memperbaiki akurasi, notebook ini menambahkan model pembandingan yang lebih kuat untuk dataset kecil, yaitu **feature-based token classifier**.

Model ini tetap bekerja pada level token NER. Fitur yang digunakan meliputi token saat ini, prefix/suffix, bentuk token, konteks kiri-kanan, bigram/trigram token, serta majority-tag gazetteer dari data train. Model yang digunakan adalah `SGDClassifier` linear. Pendekatan ini sering lebih stabil untuk dataset kecil dibanding deep learning sederhana yang belum memiliki pretrained embedding.

CELL 057 [CODE]

```

from sklearn.feature_extraction import DictVectorizer
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Train final model pada train + validation setelah arsitektur/fitur dipilih.
# Test set tetap hanya digunakan untuk evaluasi akhir.
final_train_rows = train_rows + val_rows

majority_counts = defaultdict(Counter)
for row in final_train_rows:
    for tok, tag in zip(row['tokens'], row['ner_tags']):
        majority_counts[tok.lower()][tag] += 1
majority_tag = {tok: cnt.most_common(1)[0][0] for tok, cnt in majority_counts.items()}

def token_shape(token):
    parts = []
    for ch in str(token):
        if ch.isdigit():
            c = 'd'
        elif ch.isalpha():
            c = 'a'
        else:
            c = 'p'
        if not parts or parts[-1] != c:
            parts.append(c)
    return ''.join(parts)

def feature_token(tokens, i):
    token = str(tokens[i]).lower()
    n = len(tokens)
    feats = {
        'bias': 1,
        'tok=' + token: 1,
        'majority=' + majority_tag.get(token, 'UNK'): 1,
        'prefix1=' + token[:1]: 1,
        'prefix2=' + token[:2]: 1,
        'prefix3=' + token[:3]: 1,
        'prefix4=' + token[:4]: 1,
        'suffix1=' + token[-1:]: 1,
        'suffix2=' + token[-2:]: 1,
        'suffix3=' + token[-3:]: 1,
        'suffix4=' + token[-4:]: 1,
        'shape=' + token_shape(token): 1,
        'len=' + str(min(len(token), 12)): 1,
        'has_digit=' + str(any(ch.isdigit() for ch in token)): 1,
        'pos_bin=' + str(int(i / max(1, n - 1) * 4)): 1,
    }

```

```

for off in [-3, -2, -1, 1, 2, 3]:
    j = i + off
    if 0 <= j < n:
        ctx = str(tokens[j]).lower()
        feats[f'w{off}=' + ctx] = 1
        feats[f'majority{off}=' + majority_tag.get(ctx, 'UNK')] = 1
    else:
        feats[f'edge{off}'] = 1
if i > 0:
    feats['prev_cur=' + str(tokens[i-1]).lower() + '_' + token] = 1
if i < n - 1:
    feats['cur_next=' + token + '_' + str(tokens[i+1]).lower()] = 1
if 0 < i < n - 1:
    feats['tri=' + str(tokens[i-1]).lower() + '_' + token + '_' + str(tokens[i+1]).lower()] = 1
return feats

```

```

def rows_to_xy_feature(rows):
    X, y, meta = [], [], []
    for row_id, row in enumerate(rows):
        tokens = row['tokens']
        for i, tag in enumerate(row['ner_tags']):
            X.append(feature_token(tokens, i))
            y.append(tag)
            meta.append((row_id, i, tokens[i]))
    return X, y, meta

```

```

X_train_feat, y_train_feat, _ = rows_to_xy_feature(final_train_rows)
X_val_feat, y_val_feat, _ = rows_to_xy_feature(val_rows)
X_test_feat, y_test_feat, test_meta_feat = rows_to_xy_feature(test_rows)

```

```

vectorizer_feat = DictVectorizer(sparse=True)
X_train_vec = vectorizer_feat.fit_transform(X_train_feat)
X_val_vec = vectorizer_feat.transform(X_val_feat)
X_test_vec = vectorizer_feat.transform(X_test_feat)

```

```

print('Shape fitur train:', X_train_vec.shape)
print('Jumlah tag:', len(all_tags))

```

CELL 058 [MARKDOWN]

25.1 Training model revisi

`SGDClassifier` dipilih karena efisien untuk fitur sparse berjumlah besar. Hyperparameter `alpha=3e-4` dipilih karena memberi keseimbangan antara performa validasi dan generalisasi. Model ini bukan pengganti penjelasan deep learning, melainkan **model pembandingan tambahan** untuk meningkatkan kualitas deployment NER pada dataset kecil.

CELL 059 [CODE]

```

def compute_non_o_metrics_sklearn(y_true, y_pred):
    non_o_labels = [tag for tag in all_tags if tag != 'O']
    p, r, f1, _ = precision_recall_fscore_support(
        y_true, y_pred, labels=non_o_labels, average='micro', zero_division=0
    )
    macro_p, macro_r, macro_f1, _ = precision_recall_fscore_support(
        y_true, y_pred, labels=non_o_labels, average='macro', zero_division=0
    )
    return {
        'micro_precision_non_o': p,
        'micro_recall_non_o': r,
        'micro_f1_non_o': f1,
        'macro_precision_non_o': macro_p,
        'macro_recall_non_o': macro_r,
        'macro_f1_non_o': macro_f1,
        'token_accuracy_all_tags': np.mean(np.array(y_true) == np.array(y_pred)),
    }

```

```

feature_ner_model = SGDClassifier(
    loss='hinge',
    alpha=3e-4,
    max_iter=30,
    tol=1e-3,
    class_weight=None,
    random_state=42,
    n_jobs=-1,
)
feature_ner_model.fit(X_train_vec, y_train_feat)

```

```

val_pred_feat = feature_ner_model.predict(X_val_vec)
test_pred_feat = feature_ner_model.predict(X_test_vec)

```

```

feature_val_metrics = compute_non_o_metrics_sklearn(y_val_feat, val_pred_feat)
feature_test_metrics = compute_non_o_metrics_sklearn(y_test_feat, test_pred_feat)

```

test_metrics_full berasal dari evaluasi BiLSTM pada bagian sebelumnya.

```

feature_comparison_df = pd.DataFrame([
    {'model': 'BiLSTM token classification', 'split': 'test', 'micro_f1_non_o':
test_metrics_full['micro_f1_non_o'], 'macro_f1_non_o': test_metrics_full['macro_f1_non_o']},
    {'model': 'Feature_SGD-TokenClassifier', 'split': 'validation', 'micro_f1_non_o':
feature_val_metrics['micro_f1_non_o'], 'macro_f1_non_o': feature_val_metrics['macro_f1_non_o']},
    {'model': 'Feature_SGD-TokenClassifier', 'split': 'test', 'micro_f1_non_o':

```

```
feature_test_metrics['micro_f1_non_o'], 'macro_f1_non_o': feature_test_metrics['macro_f1_non_o']},
])
display(feature_comparison_df)
```

CELL 060 [MARKDOWN]

25.2 Evaluasi dan error analysis model revisi

Evaluasi tetap menggunakan precision, recall, dan F1 token-level non-0. Token `0` tidak dijadikan fokus utama karena tujuan NER adalah mengenali token entitas/aspek. Selain itu, prediksi disimpan ke dataframe agar kesalahan boundary dan kesalahan label dapat diperiksa.

CELL 061 [CODE]

```
pd.DataFrame(classification_report(y_test_feat, test_pred_feat, labels=all_tags, output_dict=True,
zero_division=0)).T.to_csv(RESULTS_DIR / 'ner_classification_report_token_level.csv')
with open(RESULTS_DIR / 'ner_classification_report_token_level.txt', 'w', encoding='utf-8') as f:
    f.write(classification_report(y_test_feat, test_pred_feat, labels=all_tags, zero_division=0))

pd.DataFrame(confusion_matrix(y_test_feat, test_pred_feat, labels=all_tags), index=all_tags,
columns=all_tags).to_csv(RESULTS_DIR / 'ner_confusion_matrix_token_level.csv')

pred_rows = []
for (row_id, token_idx, token), gold, pred in zip(test_meta_feat, y_test_feat, test_pred_feat):
    pred_rows.append({
        'row_id': row_id,
        'token_index': token_idx,
        'token': token,
        'true_tag': gold,
        'pred_tag': pred,
        'correct': gold == pred,
    })
feature_pred_df = pd.DataFrame(pred_rows)
feature_pred_df.to_csv(RESULTS_DIR / 'ner_token_predictions_test.csv', index=False)
feature_pred_df[feature_pred_df['true_tag'] != feature_pred_df['pred_tag']].head(300).to_csv(RESULTS_DIR
/ 'ner_error_examples.csv', index=False)

display(feature_pred_df[feature_pred_df['true_tag'] != feature_pred_df['pred_tag']].head(15))
```

CELL 062 [MARKDOWN]

25.3 Simpan model NER terbaik untuk Streamlit

Model BiLSTM tetap dipertahankan sebagai model deep learning sesuai ketentuan soal. Namun, untuk deployment akhir, model NER terbaik yang digunakan adalah `Feature\_SGD-TokenClassifier` karena nilai micro-F1 non-0 pada test set lebih tinggi. Payload model menyimpan vectorizer, classifier, daftar tag, dan majority-tag gazetteer agar aplikasi Streamlit dapat melakukan ekstraksi NER pada input baru.

CELL 063 [CODE]

```
feature_payload = {
    'model_type': 'Feature_SGD-TokenClassifier',
    'vectorizer': vectorizer_feat,
    'classifier': feature_ner_model,
    'majority_tag': majority_tag,
    'all_tags': all_tags,
    'non_o_labels': [tag for tag in all_tags if tag != '0'],
    'feature_note': 'Token identity, prefix/suffix, token shape, left-right context, bigram/trigram
context, and majority-tag gazetteer.',
    'test_metrics': feature_test_metrics,
}
joblib.dump(feature_payload, MODELS_DIR / 'Kelp2_best_ner_model.joblib')
joblib.dump(feature_payload, MODELS_DIR / 'Kelp2_best_ner_feature_model.joblib')

with open(RESULTS_DIR / 'ner_best_test_metrics.json', 'w', encoding='utf-8') as f:
    json.dump({'model': 'Feature_SGD-TokenClassifier', 'split': 'test', **feature_test_metrics}, f,
indent=2)

feature_comparison_df.to_csv(RESULTS_DIR / 'ner_experiment_comparison.csv', index=False)
print('Model NER revisi berhasil disimpan untuk Streamlit.')
```

CELL 064 [MARKDOWN]

25.4 Interpretasi peningkatan akurasi NER

Performa NER meningkat karena model revisi memanfaatkan fitur token dan konteks yang eksplisit. Pada dataset kecil, informasi seperti token tetangga, prefix/suffix, dan majority tag sering sangat membantu mengenali aspek. Error yang masih tersisa terutama berupa boundary error, misalnya model hanya mengambil sebagian frasa, serta label error pada kata yang bisa bermakna produk, tempat, atau layanan tergantung konteks.

FILE: streamlit/Kelp2_app.py

```
import html
import re
from pathlib import Path

import joblib
import pandas as pd
import streamlit as st

ROOT = Path(__file__).resolve().parents[1]
ML_MODEL_PATH = ROOT / "models" / "Kelp2_best_multilabel_model.joblib"
NER_MODEL_PATH = ROOT / "models" / "Kelp2_best_ner_model.joblib"
```

```

def clean_text(text: str) -> str:
    text = str(text).lower()
    text = re.sub(r"http\S+|www\.\S+", " ", text)
    text = re.sub(r"^\0-9a-zA-Z_\-\s]", " ", text)
    return re.sub(r"\s+", " ", text).strip()

def token_shape(token: str) -> str:
    parts = []
    for ch in str(token):
        if ch.isdigit():
            cls = "d"
        elif ch.isalpha():
            cls = "a"
        else:
            cls = "p"
        if not parts or parts[-1] != cls:
            parts.append(cls)
    return "".join(parts)

def token_features(tokens, i, majority_tag):
    token = str(tokens[i]).lower()
    n = len(tokens)
    feats = {
        "bias": 1,
        "tok=" + token: 1,
        "majority=" + majority_tag.get(token, "UNK"): 1,
        "prefix1=" + token[:1]: 1,
        "prefix2=" + token[:2]: 1,
        "prefix3=" + token[:3]: 1,
        "prefix4=" + token[:4]: 1,
        "suffix1=" + token[-1:]: 1,
        "suffix2=" + token[-2:]: 1,
        "suffix3=" + token[-3:]: 1,
        "suffix4=" + token[-4:]: 1,
        "shape=" + token_shape(token): 1,
        "len=" + str(min(len(token), 12)): 1,
        "has_digit=" + str(any(ch.isdigit() for ch in token)): 1,
        "pos_bin=" + str(int(i / max(1, n - 1) * 4)): 1,
    }
    for off in [-3, -2, -1, 1, 2, 3]:
        j = i + off
        if 0 <= j < n:
            ctx = str(tokens[j]).lower()
            feats[f"w{off}=" + ctx] = 1
            feats[f"majority{off}=" + majority_tag.get(ctx, "UNK")] = 1
        else:
            feats[f"edge{off}"] = 1
    if i > 0:
        feats["prev_cur=" + str(tokens[i - 1]).lower() + "_" + token] = 1
    if i < n - 1:
        feats["cur_next=" + token + "_" + str(tokens[i + 1]).lower()] = 1
    if 0 < i < n - 1:
        feats["tri=" + str(tokens[i - 1]).lower() + "_" + token + "_" + str(tokens[i + 1]).lower()] = 1
    return feats

@st.cache_resource
def load_multilabel_model():
    return joblib.load(ML_MODEL_PATH)

@st.cache_resource
def load_ner_model():
    return joblib.load(NER_MODEL_PATH)

def predict_multilabel(text: str):
    payload = load_multilabel_model()
    model, labels = payload["model"], payload["labels"]
    x = [clean_text(text)]
    thresholds = payload.get("thresholds")

    try:
        scores = model.predict_proba(x)[0]
        if thresholds is not None:
            pred = [(score >= threshold) for score, threshold in zip(scores, thresholds)]
            if not any(pred):
                pred[int(max(range(len(scores)), key=lambda idx: scores[idx]))] = True
        else:
            pred = model.predict(x)[0]
    except Exception:
        scores = [None] * len(labels)
        pred = model.predict(x)[0]

    rows = []
    for label, active, score in zip(labels, pred, scores):
        if int(active) == 1:

```

```

        rows.append({"label": label, "confidence": None if score is None else round(float(score),
4)}})
    return rows or [{"label": "OUT_OF_TOPIC", "confidence": None}]

def predict_ner(text: str):
    payload = load_ner_model()
    tokens = clean_text(text).split()
    if not tokens:
        return []
    feats = [token_features(tokens, i, payload["majority_tag"]) for i in range(len(tokens))]
    X = payload["vectorizer"].transform(feats)
    tags = payload["classifier"].predict(X)
    return [{"token": token, "tag": tag} for token, tag in zip(tokens, tags)]

def group_entities(token_tags):
    entities, current = [], None
    for item in token_tags:
        token, tag = item["token"], item["tag"]
        if tag == "O":
            if current:
                entities.append(current)
                current = None
            continue
        prefix, label = tag.split("-", 1) if "-" in tag else ("B", tag)
        if prefix == "B" or current is None or current["label"] != label:
            if current:
                entities.append(current)
            current = {"label": label, "text": token}
        else:
            current["text"] += " " + token
    if current:
        entities.append(current)
    return entities

def render_highlight(token_tags):
    chunks = []
    for item in token_tags:
        token = html.escape(item["token"])
        tag = item["tag"]
        if tag == "O":
            chunks.append(token)
        else:
            label = html.escape(tag.split("-", 1)[1] if "-" in tag else tag)
            chunks.append(f"<mark title='{label}' style='padding:2px 4px; border-
radius:4px'>{token}</mark>")
    return " ".join(chunks)

st.set_page_config(page_title="UAS PBA ABSA", layout="wide")
st.title("UAS PBA - ABSA Multilabel Text Classification & NER")
st.caption("Kelompok 2 | Dataset lanjutan UTS Google Places Review")

text = st.text_area(
    "Masukkan review Google Places:",
    value="Tempatnya nyaman, bajunya bagus, harga terjangkau, tapi parkirnya kurang luas.",
    height=120,
)

tab1, tab2, tab3 = st.tabs(["Prediksi Multilabel ABSA", "Prediksi NER ABSA", "Hubungan Output"])

with tab1:
    st.subheader("Prediksi Label Review")
    if st.button("Prediksi Multilabel"):
        st.dataframe(pd.DataFrame(predict_multilabel(text)), use_container_width=True)
        st.info("Multilabel menunjukkan aspek dan sentimen pada level review. Satu review dapat memiliki
lebih dari satu label.")

with tab2:
    st.subheader("Ekstraksi Span/Aspek")
    if st.button("Prediksi NER"):
        token_tags = predict_ner(text)
        st.markdown(render_highlight(token_tags), unsafe_allow_html=True)
        st.write("Entitas/Aspek yang terdeteksi:")
        st.dataframe(pd.DataFrame(group_entities(token_tags)), use_container_width=True)
        st.write("Detail token-tag:")
        st.dataframe(pd.DataFrame(token_tags), use_container_width=True)

with tab3:
    st.subheader("Hubungan Output Multilabel dan NER")
    st.write(
        "Multilabel memutuskan aspek-sentimen pada level review, sedangkan NER menunjukkan span/token
yang menjadi bukti aspek tersebut. "
        "Contoh: PRODUCT_POSITIVE dapat didukung oleh span 'bajunya bagus', PRICE_POSITIVE oleh 'harga
terjangkau', "
        "dan PLACE_POSITIVE oleh 'tempatny nyaman'."
    )

```